

Comprensione del Codice Sorgente: Un Esperimento su Commenti Umani, Commenti AI e Chatbot Conversazionali

1^a Marta Coiro
Università degli Studi di Salerno
Fisciano, Italia
m.coiro9@studenti.unisa.it

2^a Katia Buonocore
Università degli Studi di Salerno
Fisciano, Italia
k.buonocore1@studenti.unisa.it

Abstract

In questo studio esploriamo come diverse forme di documentazione del codice — commenti scritti da umani, commenti generati dall'Intelligenza Artificiale (AI) o conversazioni con un assistente virtuale — influenzino la comprensione del codice sorgente da parte degli studenti universitari. Attraverso un esperimento condotto con 18 partecipanti, abbiamo analizzato l'efficacia percepita e reale di ciascun approccio. Il nostro sistema web-based ha somministrato frammenti in Java e Rust, misurando comprensione, preferenze e tempi di risposta. I risultati rivelano schemi interessanti nelle preferenze individuali e nell'efficacia oggettiva dei diversi approcci.

Parole Chiave — commenti, AI, assistente virtuale

1. INTRODUZIONE

Comprendere il codice sorgente rappresenta una delle sfide più significative per gli sviluppatori software, sia in ambito accademico che industriale. Nonostante la crescente automazione nei processi di sviluppo, gran parte del lavoro di un programmatore consiste ancora nel leggere, comprendere e modificare codice scritto da altri. In questo contesto, la documentazione svolge un ruolo centrale, fungendo da supporto cognitivo per facilitare la decodifica delle intenzioni progettuali originali e prevenire errori di interpretazione.

Tradizionalmente, la comprensione è supportata da commenti scritti a mano da sviluppatori esperti. Tuttavia, l'avvento delle tecnologie di Intelligenza Artificiale ha introdotto nuovi modi per supportare la lettura del codice: dalla generazione automatica di riassunti e descrizioni fino alla possibilità di interagire direttamente con agenti conversazionali in grado di rispondere a domande ed esporre spiegazioni sui frammenti di codice in tempo reale.

Questi nuovi metodi promettono di aumentare l'accessibilità del codice, soprattutto per studenti o programmatori alle prime armi. Tuttavia, rimane una domanda fondamentale: quale di queste strategie è davvero più efficace nel facilitare la comprensione? Gli utenti preferiscono il supporto statico fornito dai commenti (siano essi umani o generati automaticamente), oppure traggono maggior beneficio dall'interazione attiva con un assistente conversazionale?

In questo articolo proponiamo un'indagine sistematica condotta attraverso un esperimento controllato, con l'obiettivo di confrontare direttamente tre metodi di supporto alla comprensione del codice:

- Commenti scritti da esseri umani;
- Commenti generati da tecniche di sintesi basate sull'Intelligenza Artificiale;
- Conversazione con un chatbot basato su modelli linguistici di grandi dimensioni (LLM).

L'obiettivo è analizzare l'impatto di ciascun metodo in termini di accuratezza delle risposte, tempi di lettura, preferenze soggettive e modalità di interazione. Lo studio è rivolto a studenti universitari, offrendo quindi spunti utili sia per l'insegnamento della programmazione sia per lo sviluppo di strumenti di supporto allo sviluppo software.

2. LAVORI CORRELATI

Negli ultimi anni, la crescente accessibilità e le capacità dei **Large Language Models (LLM)** hanno stimolato un numero sempre maggiore di ricerche su come questi strumenti possano supportare la comprensione del codice sorgente. Diversi studi hanno indagato l'efficacia sia delle spiegazioni scritte da esseri umani che di quelle generate dall'Intelligenza Artificiale, oltre ad approcci interattivi come gli agenti conversazionali.

Stapleton et al. (2020) [1] hanno condotto uno dei primi studi su larga scala confrontando riassunti scritti da esseri umani con quelli generati automaticamente. I risultati hanno mostrato che i commenti umani migliorano significativamente la comprensione del codice ($p = 0,029$) rispetto ad approcci automatici basati su AST e modelli sequenziali.

Wu et al. (2024) [2] hanno proposto **CODERPE**, un valutatore basato su LLM che ha superato le metriche automatiche tradizionali (BLEU, ROUGE) nella valutazione dei riassunti di codice, evidenziando il potenziale degli LLM non solo per la generazione ma anche per la valutazione.

Leinonen et al. (2023) [3] hanno confrontato spiegazioni scritte da studenti con quelle generate da GPT-3 e GPT-4. I risultati hanno mostrato che i partecipanti generalmente preferivano le spiegazioni generate dagli LLM, in particolare per chiarezza e sinteticità. Questo suggerisce un potenziale crescente per l'integrazione degli LLM negli strumenti educativi. Allo stesso modo, **Sun et al. (2024)** [4] hanno condotto una valutazione

automatica utilizzando GPT-4 e CodeLlama-Instruct, dimostrando che modelli open-source ottimizzati possono superare quelli generici.

Infine, **Nam et al. (2024)** [5] hanno esplorato un approccio ibrido, integrando l'assistenza basata su LLM in un plugin per IDE. Il loro studio con utenti ha dimostrato che l'ambiente potenziato da LLM ha migliorato i tempi di completamento dei compiti, l'accuratezza e l'utilità percepita, soprattutto quando gli utenti potevano interagire con il modello in linguaggio naturale.

Mentre questi studi si sono concentrati principalmente sul confronto tra diversi tipi di commenti o sulla valutazione della qualità della sintesi, il nostro lavoro aggiunge una nuova dimensione confrontando tre modalità distinte — commenti umani, commenti generati dall'AI e assistenza interattiva tramite chatbot — in uno studio controllato sugli utenti. Inoltre, il nostro disegno controbilanciato garantisce un confronto equo tra linguaggi e ordini di presentazione, un fattore spesso trascurato nelle ricerche precedenti.

3. LA PIATTAFORMA CODEINSIGHT

La piattaforma **CodeInsight** è una piattaforma web personalizzata sviluppata per condurre il nostro esperimento di comprensione del codice, che coinvolge tre diverse modalità di documentazione: commenti scritti da esseri umani, riassunti generati dall'AI e interazione con un chatbot.

La piattaforma è stata costruita utilizzando **Flask (Python)** per il backend e **SQLite** per l'archiviazione persistente, mentre il frontend è stato sviluppato con **HTML, CSS e JavaScript** standard.

La piattaforma è composta da due componenti principali:

- **Il Visualizzatore di Codice**, che mostra frammenti di codice in Java o Rust, con una delle tre modalità di documentazione a seconda della condizione sperimentale assegnata al partecipante. Il visualizzatore include l'evidenziazione della sintassi per migliorare la leggibilità e il coinvolgimento durante la fase di comprensione.
- **L'Area di Interazione**, che si adatta dinamicamente alla condizione assegnata:
 - Nelle modalità **Commenti Umani (HC)** e **Commenti AI (AIC)**, un blocco di commento precompilato è mostrato direttamente sopra il codice.
 - Nella modalità **Chatbot (Chat)**, il codice appare senza alcun commento, ma l'utente può interagire con un chatbot basato su LLM (alimentato da **OpenAI GPT-4o-mini**) per fare domande prima di iniziare il quiz. Questa interfaccia di chat è posizionata appena sotto la sezione del quiz e diventa di sola lettura dopo l'inizio del quiz, preservando il contesto ma impedendo ulteriori interazioni.

Ogni frammento di codice è seguito da una sezione di quiz, composta da:

- due domande a scelta multipla,
- una domanda a risposta aperta per valutare la comprensione del codice,
- e, solo per i codici con commenti umani, una domanda aggiuntiva a scelta multipla per chiedere all'utente se i commenti siano scritti da umani oppure no.

L'interfaccia è progettata per garantire un'esperienza d'uso fluida: i partecipanti possono navigare liberamente tra le domande del quiz utilizzando i pulsanti *Next* e *Previous*. Tuttavia, le risposte possono essere inviate solo collettivamente al termine di ogni sessione di snippet tramite un pulsante dedicato *Submit Quiz*. Prima dell'avvio del quiz viene mostrata una finestra di conferma per assicurarsi che l'utente sia pronto.

Tutte le azioni degli utenti — come l'apertura di uno snippet, l'interazione con il chatbot, la navigazione tra le domande e l'invio delle risposte — vengono registrate con timestamp e salvate nel database per l'analisi successiva. La piattaforma è basata su sessioni e completamente anonimizzata, utilizzando ID manuali non collegati a e-mail istituzionali, al fine di garantire la privacy e la conformità al GDPR.

CodeInsight supporta:

- il salvataggio in tempo reale delle risposte,
- il caricamento strutturato dei dati da un dataset JSON predefinito,
- una matrice di assegnazione bilanciata degli snippet, che garantisce a ciascun partecipante una combinazione unica ed equilibrata di tipi di documentazione in Java e Rust.

Esperimento: Comprensione del codice

Benvenuto! Stai partecipando a uno studio che esplora diversi modi di interpretare il codice sorgente. Ti mostreremo frammenti di codice in tre varianti:

- Con commenti umani.
- Con commenti generati dall'Intelligenza Artificiale.
- Possibilità di interagire con un assistente virtuale.

Tutte le informazioni raccolte saranno utilizzate esclusivamente per scopi di ricerca e saranno trattate in modo anonimo.

ID Manuale

Inizia

Fig. 1: "Welcome" home page

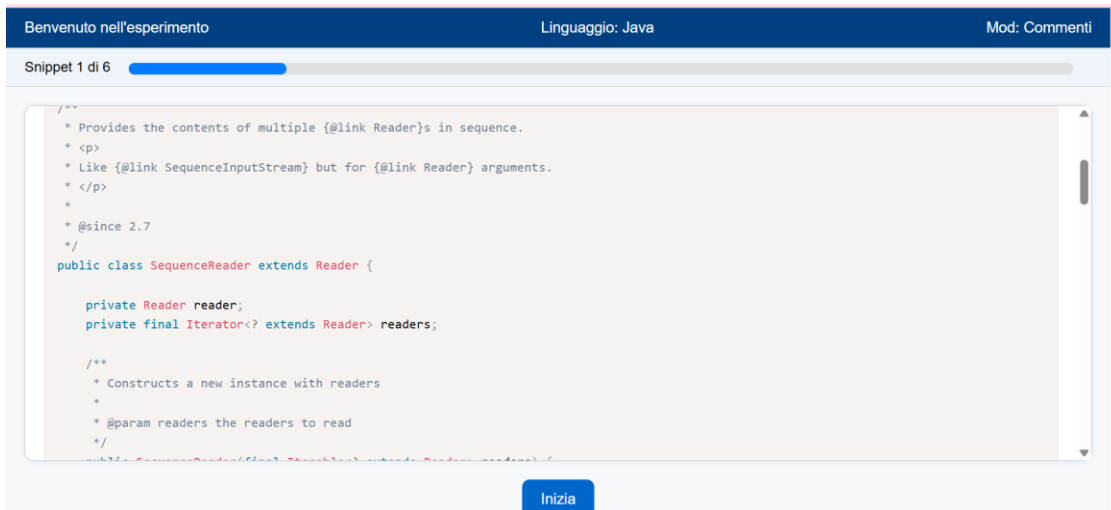


Fig. 2: View snippets in one of three modes

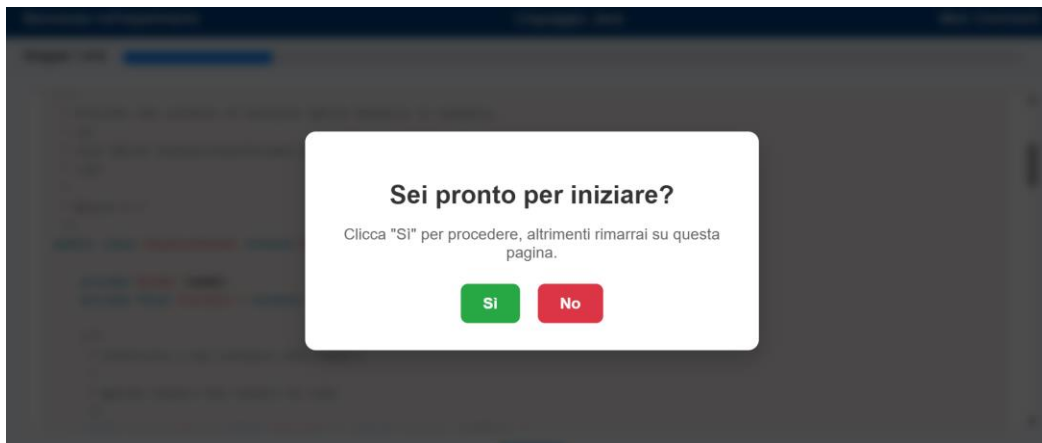


Fig. 3: Experiment start confirmation banner

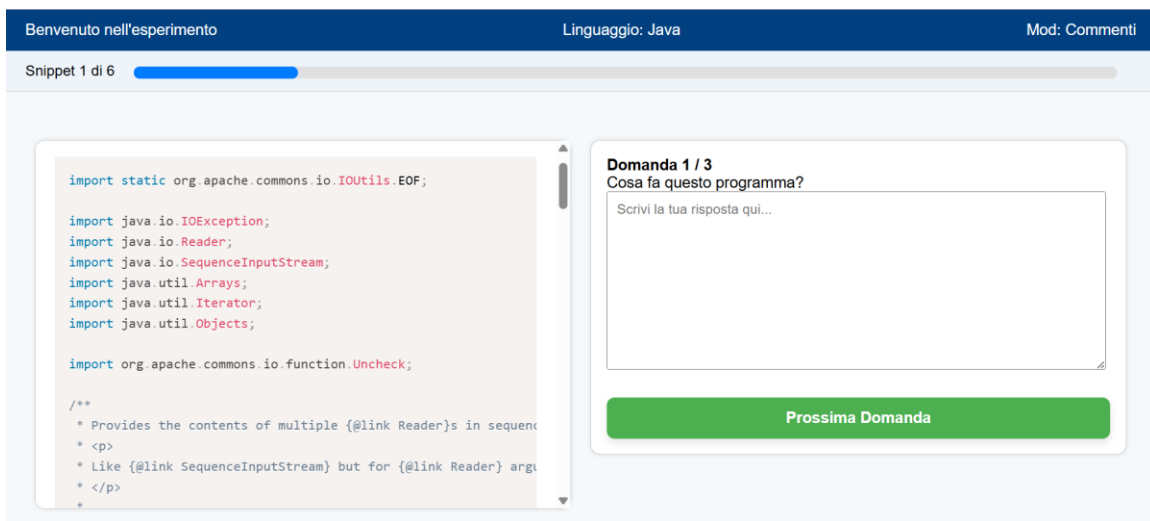


Fig. 4: Open question display

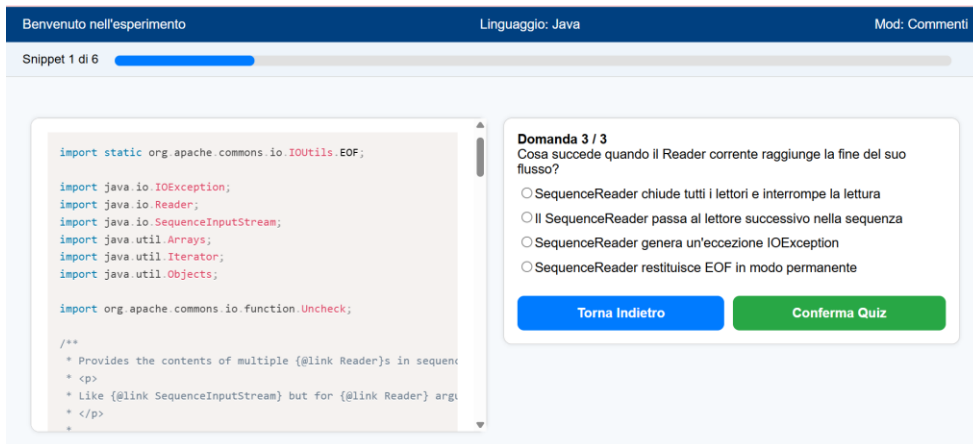


Fig.5: Viewingquizdisplay

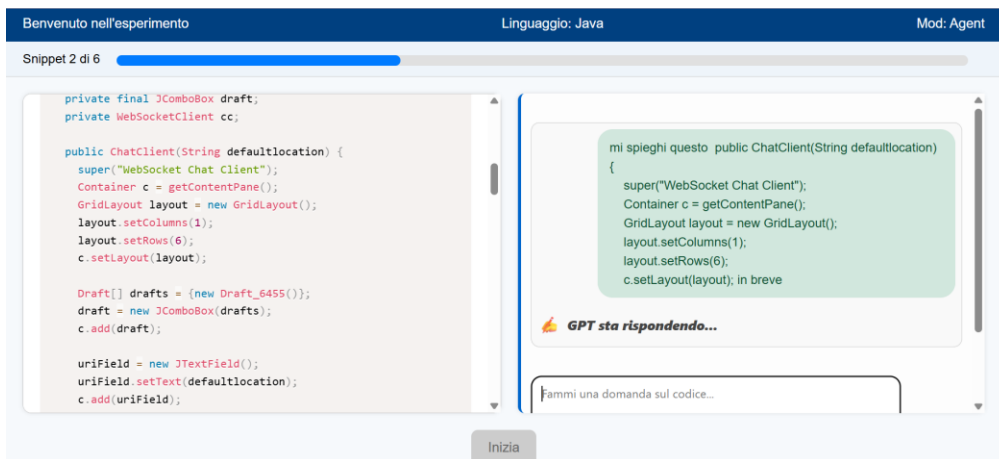


Fig.6: ViewingchatwithChat-gpt



Fig.7: Finalpage

4. STUDIO CON GLI UTENTI

Questo esperimento mira a rispondere alle seguenti domande di ricerca:

1. I partecipanti comprendono meglio il codice quando supportati da commenti scritti da esseri umani, riassunti generati dall'AI o da un agente conversazionale?
2. Quale modalità di documentazione porta a prestazioni migliori nei quiz (accuratezza e tempo)?
3. Quali sono i vantaggi e i limiti percepiti di ciascun approccio dal punto di vista dei partecipanti?

Per esplorare queste domande, abbiamo condotto uno **studio con 18 partecipanti** utilizzando la piattaforma CodeInsight.

Ogni partecipante ha ricevuto **6 frammenti di codice**: 3 in Java e 3 in Rust, distribuiti nelle tre modalità di documentazione (**Commenti Umani – HC, Commenti AI – AIC e Interazione con Chatbot – Chat**).

Ogni sessione includeva attività strutturate di quiz e questionari di valutazione.

4.1 Partecipanti

Hanno partecipato **18 studenti** (15 uomini e 3 donne), di età compresa tra 21 e 26 anni, su base volontaria.

Tutti erano iscritti a corsi di Laurea o Laurea Magistrale in Informatica o Ingegneria Informatica.

I partecipanti avevano background eterogenei ma una conoscenza comune dei concetti base di programmazione.

A ciascuno è stato assegnato un **ID univoco manuale** per accedere all'esperimento in forma anonima.

Tabella I – Dettagli dei partecipanti

Partecipanti M F NB Età media

18	15	3	–	23,78 (DS 1,66)
----	----	---	---	-----------------

4.2 Disegno sperimentale

L'esperimento segue un **disegno entro-soggetti**, in cui tutti i partecipanti sono esposti alle tre modalità di documentazione.

Ogni partecipante ha visualizzato **uno snippet in Java e uno in Rust** per ciascuna delle tre

condizioni (HC, AIC, Chat), seguendo una matrice di assegnazione **pienamente controbilanciata** (Tabella II).

TABELLA II: Fig. 1. Assegnazione dei frammenti Java e Rust ai partecipanti nelle tre modalità (AIC, HC, Chat).

Partecipante	Java			Rust		
	AIC	HC	Chat	AIC	HC	Chat
1	TJ1	TJ2	TJ3	TR1	TR2	TR3
2	TJ2	TJ3	TJ1	TR2	TR3	TR1
3	TJ3	TJ1	TJ2	TR3	TR1	TR2
	HC	Chat	AIC	HC	Chat	AIC
4	TJ1	TJ2	TJ3	TR1	TR2	TR3
5	TJ2	TJ3	TJ1	TR2	TR3	TR1
6	TJ3	TJ1	TJ2	TR3	TR1	TR2
	Chat	AIC	HC	Chat	AIC	HC
7	TJ1	TJ2	TJ3	TR1	TR2	TR3
8	TJ2	TJ3	TJ1	TR2	TR3	TR1
9	TJ3	TJ1	TJ2	TR3	TR1	TR2
Rust ↔ Java						
	AIC	HC	Chat	AIC	HC	Chat
10	TR1	TR2	TR3	TJ1	TJ2	TJ3
11	TR2	TR3	TR1	TJ2	TJ3	TJ1
12	TR3	TR1	TR2	TJ3	TJ1	TJ2
	HC	Chat	AIC	HC	Chat	AIC
13	TR1	TR2	TR3	TJ1	TJ2	TJ3
14	TR2	TR3	TR1	TJ2	TJ3	TJ1
15	TR3	TR1	TR2	TJ3	TJ1	TJ2
	Chat	AIC	HC	Chat	AIC	HC
16	TR1	TR2	TR3	TJ1	TJ2	TJ3
17	TR2	TR3	TR1	TJ2	TJ3	TJ1
18	TR3	TR1	TR2	TJ3	TJ1	TJ2

4.3 Apparato

L'esperimento è stato condotto utilizzando la piattaforma **CodeInsight**.

Ogni partecipante ha avuto accesso all'esperimento tramite un browser web su computer di laboratorio.

I computer del laboratorio erano dotati di:

- CPU **Intel® Core™ i5-7200U** (2.50–2.70 GHz),
- **16 GB RAM**,
- **Windows 10 Pro 64-bit** (versione 22H2, build 19045.6216).

Ogni postazione era collegata a un monitor **Philips 275S9JAL/00 27" QHD LCD** (risoluzione 2560×1440, tecnologia IPS, tempo di risposta 4 ms, luminosità 350 cd/m²), con funzioni ergonomiche e tecnologia low-blue light.

Il sistema tracciava tutte le interazioni dell'utente, inclusi:

- timestamp per la visualizzazione del codice,

- interazione con la chat,
- risposte ai quiz.

L'interfaccia di chat era disponibile solo per i frammenti in modalità Chat ed era **bloccata una volta iniziato il quiz**.

4.4 Procedura

All'inizio della sessione, a ciascun partecipante veniva fornito un codice di accesso univoco e una spiegazione introduttiva sugli obiettivi dello studio.

I partecipanti dovevano compilare un questionario iniziale contenente:

- consenso al trattamento dei dati (GDPR 2016/679),
- ID, età, genere, livello di istruzione e corso di laurea,
- esperienza di programmazione,
- livello di competenza in **Java** e **Rust** (scala Likert 1–5),
- frequenza d'uso degli LLM,
- uso degli LLM per compiti di programmazione.

Prima di iniziare, i partecipanti hanno visto una **breve demo** che illustrava il funzionamento dell'esperimento.

Fase di esecuzione dei compiti

- Ogni partecipante ha lavorato su 6 compiti (uno per frammento).
- Per ciascun compito, veniva mostrato un frammento di codice in una delle tre modalità (HC, AIC o Chat).
- In modalità **Chat**, i partecipanti potevano interagire con l'assistente conversazionale **prima** di iniziare il quiz. Dopo l'avvio del quiz, la chat diventava sola lettura.
- Ogni compito aveva una durata massima di **8 minuti**: fino a 6 minuti per la lettura del codice e ~2 minuti per rispondere alle domande.
- Se la lettura veniva completata in anticipo, il tempo rimanente veniva aggiunto al tempo per le risposte.

Quiz per ogni frammento:

- 2 domande a scelta multipla (con un'ulteriore domanda per distinguere commenti umani da AI nei casi HC e AIC),

- 1 domanda a risposta aperta.

Gli utenti potevano navigare tra le domande con i pulsanti *Next* e *Back*. Una finestra di conferma era mostrata prima dell'inizio di ogni quiz. Tutte le azioni e i tempi venivano registrati automaticamente.

Valutazione post-attività

Dopo i sei snippet, i partecipanti hanno ricevuto un riepilogo finale che mostrava, per ciascun frammento, se i commenti fossero umani o AI.

Successivamente, hanno compilato **tre questionari ispirati al NASA-TLX**, uno per ciascuna modalità.

Infine, hanno scritto commenti liberi per valutare usabilità e soddisfazione, tra cui:

- classifica di preferenza delle modalità,
- utilità percepita per la comprensione,
- motivazioni aperte per la modalità più utile,
- opinioni sul tempo concesso,
- osservazioni aggiuntive.

L'intera sessione è durata circa **30 minuti** per partecipante.

4.5 Dataset

Abbiamo costruito manualmente un dataset di **12 frammenti di codice**: 6 in Java e 6 in Rust.

- Gli snippet Java sono stati estratti dal progetto **OpenCms** [6].
- Gli snippet Rust sono stati selezionati da repository **open-source Rust** [7].

Ogni frammento è stato duplicato in tre versioni:

- con **commenti scritti da esperti umani (HC)**, progettati per spiegare chiaramente l'intento logico e funzionale;
- con **commenti generati automaticamente da un modello AI (AIC)** specializzato nella sintesi del codice (prompt adattivi con ChatGPT-4-turbo);
- **senza commenti**, ma con possibilità di interagire via chatbot (modalità Chat).

Il dataset è stato salvato in formato **JSON** e caricato dinamicamente dal sistema sperimentale.

Le versioni chatbot erano progettate per stimolare la capacità di porre domande e ricevere risposte contestualizzate, mettendo alla prova la comprensione in modo interattivo e flessibile.

5. RISULTATI

Tutti i 18 partecipanti hanno completato integralmente l'esperimento, interagendo con sei frammenti di codice ciascuno. Ogni snippet è stato presentato in una delle seguenti tre modalità di supporto alla comprensione:

- Commenti generati dall'AI,
- Commenti scritti da esseri umani,
- Interazione con un assistente conversazionale (modalità Agente/Chatbot).

L'obiettivo principale era valutare **se e come** il tipo di supporto influenzasse le prestazioni dei partecipanti. È stata condotta un'analisi della varianza a una via (**ANOVA**) per confrontare le tre condizioni rispetto a due variabili chiave:

- **Accuratezza delle risposte** (basata sulle domande a scelta multipla),
- **Tempo di comprensione** (estratto dai log di sistema).

Inoltre, sono stati analizzati:

- le risposte aperte ("Che cosa fa questo programma?"),
- la capacità di riconoscere l'origine dei commenti (umani o AI),
- l'influenza del linguaggio di programmazione (Java vs Rust).

5.1 RQ1 – Accuratezza delle risposte

Per valutare se la modalità di supporto influenzasse l'accuratezza delle risposte a scelta multipla, abbiamo analizzato il numero medio di risposte corrette e incorrette per partecipante, raggruppate per modalità e linguaggio di programmazione. I risultati sono mostrati nella Tabella III.

I risultati mostrano piccole differenze tra le condizioni. È stata effettuata un'ANOVA a una via per determinare se queste differenze fossero statisticamente significative. L'esito è stato: $F(2, 51) = 0.375$, $p = 0.689$. Il tipo di supporto alla comprensione (IA, umano o chatbot) non ha influito in modo significativo sull'accuratezza complessiva degli utenti nelle domande a scelta multipla.

Sebbene non statisticamente significative, emergono alcuni schemi descrittivi:

- Con Java, le migliori prestazioni sono state osservate nella condizione con commenti umani (1,44 risposte corrette per utente, 2,7% del totale).
- Con Rust, gli utenti hanno ottenuto risultati leggermente migliori con commenti generati dall'IA (0,89 corretti/utente, 1,6%) rispetto ai commenti umani (0,72, 1,3%).
- La modalità Agent ha prodotto risultati intermedi con Java (1,17 corretti/utente, 2,2%), ma è stata la meno efficace con Rust (0,83 corretti/utente, 1,5%).

Queste tendenze suggeriscono che, pur mantenendosi relativamente stabile, l'accuratezza complessiva può essere influenzata da sottili differenze legate al linguaggio di programmazione e alla familiarità, influenzando potenzialmente i risultati di comprensione.

5.2. RQ2 Tempo di Comprensione

Per valutare se la modalità di supporto avesse un impatto sulla velocità con cui i partecipanti comprendevano e rispondevano al codice, è stato analizzato il tempo totale speso per ciascun frammento (comprensione + risposta), raggruppato per modalità e linguaggio di programmazione.

I risultati sono mostrati nella Tabella IV, che riporta sia il tempo totale medio che la deviazione standard. È stata condotta un'ANOVA a una via per determinare se le differenze osservate fossero statisticamente significative tra le modalità. I risultati hanno rivelato un effetto principale significativo del tipo di supporto sul tempo totale di comprensione: $F(2, 51) = 3.217, p = 0.048$

Il tipo di supporto utilizzato dai partecipanti ha influenzato in modo significativo il tempo necessario per comprendere e rispondere a ciascun frammento.

Risultati principali:

- Per Java, il tempo totale medio più breve è stato ottenuto con commenti generati dall'IA (3:25), seguito dai commenti umani (4:08) e infine dalla modalità Agent (4:21).
- Per Rust, i tempi sono stati generalmente più lunghi in tutte le modalità: i commenti umani hanno comportato il tempo totale più lungo (5:04), seguiti da Agent (4:48) e IA (4:38).

La modalità IA ha ridotto costantemente il tempo totale di comprensione, soprattutto con Java, suggerendo un processo di comprensione più efficiente in ambienti familiari. Al contrario, la modalità Agent, sebbene potenzialmente di supporto cognitivo, ha comportato tempi totali più lunghi, in particolare con Rust, dove gli utenti potrebbero aver necessitato di chiarimenti o interazioni aggiuntive per completare il compito.

5.3. RQ3 Riconoscimento dell'Origine dei Commenti

Per esplorare la capacità dei partecipanti di identificare la fonte dei commenti, è stata posta loro la domanda: “Pensi che questi commenti siano stati scritti da un essere umano?”

La Tabella V riassume il numero totale di risposte corrette e scorrette, la media per partecipante e le deviazioni standard, categorizzate per modalità e linguaggio di programmazione.

- Nella condizione di commenti generati dall'IA per **Java**, i partecipanti hanno riconosciuto correttamente l'origine non umana nel 61,1% dei casi, mentre il 38,9% credeva ancora che i commenti fossero scritti da un umano.
- Per **Rust**, il tasso di riconoscimento corretto è sceso al 43,8%, suggerendo maggior difficoltà nell'identificare contenuti generati dall'IA in basi di codice meno familiari.
- Nella condizione di commenti scritti da esseri umani, il tasso di riconoscimento più basso è stato osservato con **Java** (33,3%), indicando che i partecipanti spesso confondevano i commenti umani con quelli generati dall'IA.
- Per **Rust**, il riconoscimento è migliorato al 60,0%, suggerendo che i commenti scritti da umani erano più distinguibili.

I partecipanti hanno mostrato una capacità limitata di distinguere tra commenti umani e generati dall'IA—soprattutto in Java, dove i commenti generati dall'IA venivano più spesso scambiati per commenti umani. Ciò suggerisce che i commenti dell'IA possono imitare efficacemente la naturalezza umana in contesti familiari. Al contrario, Rust ha permesso classificazioni più accurate, in particolare per i contenuti scritti da esseri umani.

5.4. RQ4 Effetti del Linguaggio

Per valutare se il linguaggio di programmazione influenzasse le prestazioni dei partecipanti, abbiamo confrontato i risultati tra frammenti scritti in **Java** e **Rust**. La Tabella VI riporta il numero medio di risposte corrette e scorrette per partecipante, insieme alle corrispondenti deviazioni standard.

TABELLA III: Risposte corrette medie per utente (per modalità e linguaggio).

TABLE III: Average Correct Answers per User (by Mode and Language).					
Mode	Language	Total Answers	Total Correct Answers	Avg. Correct/User	Avg. Incorrect/User
agent	Java	54	21	1.17	1.83
agent	Rust	54	15	0.83	2.17
comment_ai	Java	54	19	1.06	1.94
comment_ai	Rust	54	16	0.89	2.11
comments_human	Java	54	26	1.44	1.56
comments_human	Rust	54	13	0.72	2.28

TABELLA IV: Tempo totale di comprensione + risposta (per modalità e linguaggio).

TABLE IV: Total Comprehension + Response Time (by Mode and Language)

Mode	Language	Avg. Total Time	Standard Deviation
agent	Java	4:21	1:48
agent	Rust	4:48	1:51
comment_ai	Java	3:25	1:50
comment_ai	Rust	4:38	2:34
comments_human	Java	4:08	2:10
comments_human	Rust	5:04	2:24

- Il numero medio di risposte corrette era leggermente più alto per Java (0,09 vs 0,06), sebbene la differenza sia minima.
- I partecipanti hanno fornito quasi lo stesso numero di risposte errate in entrambi i linguaggi (0,91 in Java, 0,94 in Rust).
- La variabilità tra i partecipanti era leggermente maggiore per Java, come indicato dalle deviazioni standard.

Sebbene le differenze siano piccole, i dati suggeriscono una familiarità marginalmente migliore con Java. Tuttavia, il numero di risposte errate rimane elevato in entrambi i linguaggi. Questo risultato è coerente con quanto emerso nelle risposte aperte e nei tempi di comprensione: mentre Java tende a essere elaborato leggermente più velocemente e con maggiore accuratezza, Rust può richiedere uno sforzo cognitivo maggiore, rendendo più utile la presenza di supporto conversazionale (ad esempio, la modalità Agent) in questo contesto.

5.5. Analisi complessiva delle risposte a scelta multipla

I partecipanti hanno risposto alle domande a scelta multipla con un tasso di accuratezza complessivo del 40,4% corrette e 59,6% errate, evidenziando una difficoltà generale nella comprensione del codice in tutte le condizioni di supporto.

Aggregando le risposte per modalità di supporto, la condizione con **commenti umani** ha ottenuto l'accuratezza più alta, con il 47,2% di risposte corrette. Le condizioni **commenti IA** e **Agent** hanno entrambe ottenuto un'accuratezza del 37,0%, suggerendo un'efficacia simile ma inferiore rispetto ai commenti scritti da esseri umani. Sebbene queste differenze non siano risultate statisticamente significative (come discusso in RQ1), esse evidenziano una tendenza descrittiva che potrebbe guidare future indagini.

A livello individuale, gli utenti hanno mostrato un'accuratezza media di circa il 38%, con una deviazione standard del 12%, riflettendo una variabilità moderata nelle prestazioni dei partecipanti. Alcuni utenti sono stati in grado di adattarsi più efficacemente alle diverse modalità di supporto, mentre altri hanno mantenuto un modello di risposta più stabile ma meno accurato.

In sintesi, anche se non sono state trovate differenze statisticamente significative tra le modalità di supporto, i risultati aggregati suggeriscono che i **commenti scritti da esseri umani** offrono ancora un leggero vantaggio nella facilitazione della comprensione del codice—particolarmente in contesti familiari come Java. Tuttavia, le prestazioni comparabili del supporto basato su **IA** e **chatbot** aprono interessanti prospettive per approcci ibridi per supportare la comprensione dei programmi.

5.6. NASA-TLX

L'analisi del carico mentale percepito, utilizzando il **NASA-TLX ponderato**, ha mostrato valori medi simili tra le tre modalità di supporto alla comprensione del codice:

- **Agent**: media = 10,16, deviazione standard = 3,06
- **Commenti IA**: media = 10,00, deviazione standard = 4,80
- **Commenti umani**: media = 11,06, deviazione standard = 4,60

Questi risultati suggeriscono che il carico mentale complessivamente percepito dagli utenti è relativamente bilanciato tra le modalità, con una leggera tendenza a un carico maggiore nella modalità **Commenti umani**.

Per approfondire, l'analisi delle **preferenze espresse dagli utenti** sulle 15 coppie di dimensioni TLX ha aiutato a identificare quali aspetti del carico mentale sono più influenti in ciascuna modalità. In particolare, per ogni domanda, la “dimensione vincente” è stata determinata in base al numero di preferenze degli utenti.

I grafici delle risposte vincenti mostrano schemi distinti tra le modalità:

- La modalità **Agent** tende a evidenziare le dimensioni “**Richiesta Mentale**” e “**Prestazioni**” come predominanti.
- La modalità **Commenti umani** enfatizza anche dimensioni legate a “**Frustrazione**” e “**Richiesta Temporale**”.
- La modalità **Commenti IA** mostra una distribuzione intermedia, con preferenze equilibrate tra le varie dimensioni.

Questa doppia analisi—quantitativa (TLX ponderato) e qualitativa (preferenze sulle coppie di dimensioni)—fornisce una visione più completa del carico mentale associato alle diverse

modalità di supporto. Risulta utile per guidare interventi di miglioramento e ottimizzare l'esperienza dell'utente. L'analisi offre importanti indicazioni per valutare l'efficacia e l'usabilità delle interfacce studiate.

TABELLA V: Risposte alla domanda “Questi commenti sono stati scritti da un umano?” raggruppate per modalità di supporto e linguaggio di programmazione. La tabella riporta il numero di risposte corrette, la media e la deviazione standard per partecipante.

Modality	Language	Avg. C.	SD C.
comment_ai	Java	0.611	0.502
comment_ai	Rust	0.438	0.512
comments_human	Java	0.333	0.485
comments_human	Rust	0.600	0.507

TABELLA VI: Prestazioni per linguaggio di programmazione. La tabella riporta il numero medio di risposte corrette e scorrette per utente, insieme alle deviazioni standard.

Language	Avg. Correct	SD Correct	Avg. Incorrect	SD Incorrect
Java	0.09	0.28	0.91	0.28
Rust	0.06	0.23	0.94	0.23

TABELLA VII: Accuratezza disaggregata per modalità e linguaggio.

Mode	Language	Correct	% Correct
agent	Java	21	38.9%
agent	Rust	15	27.8%
comment_ai	Java	19	35.2%
comment_ai	Rust	16	29.6%
comments_human	Java	26	48.1%
comments_human	Rust	13	24.1%

TABELLA VIII: Accuratezza aggregata per modalità di supporto.

Modality	Correct	% Correct
comments_human	51	47.2%
comment_ai	40	37.0%
agent	40	37.0%

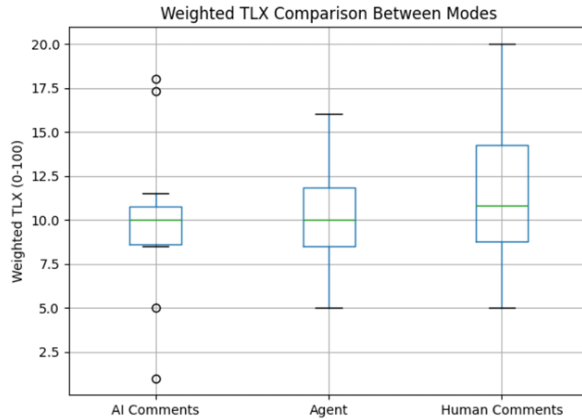


Fig. 8: Weighted TLX

5.7. Questionario finale

5.7.1. Preferenze degli utenti sulle modalità di comprensione del codice

Nei commenti liberi finali, è stato chiesto agli utenti di classificare le tre modalità di comprensione del codice in base alle loro preferenze: commenti scritti da esseri umani (HC), commenti generati automaticamente (AIC) e interazione con un assistente virtuale (Chat).

I risultati mostrano una chiara preferenza per la modalità **Chat**, che è stata classificata al primo posto da 11 partecipanti su 18. Questa modalità è stata la più apprezzata, probabilmente grazie alla sua interattività e alla possibilità di porre domande chiarificatrici in tempo reale.

La modalità **commenti generati automaticamente (AIC)** è stata selezionata più frequentemente come seconda scelta, indicando un interesse moderato ma non predominante. Infine, i **commenti scritti da esseri umani (HC)** sono stati più spesso classificati come terza scelta, risultando complessivamente la modalità meno preferita.

Questi risultati suggeriscono che, nel contesto di questo esperimento, gli utenti hanno trovato la natura dinamica e adattiva dell'assistente virtuale più efficace per comprendere il codice rispetto ad approcci più statici.

Quale modalità hai preferito tra quelle testate?

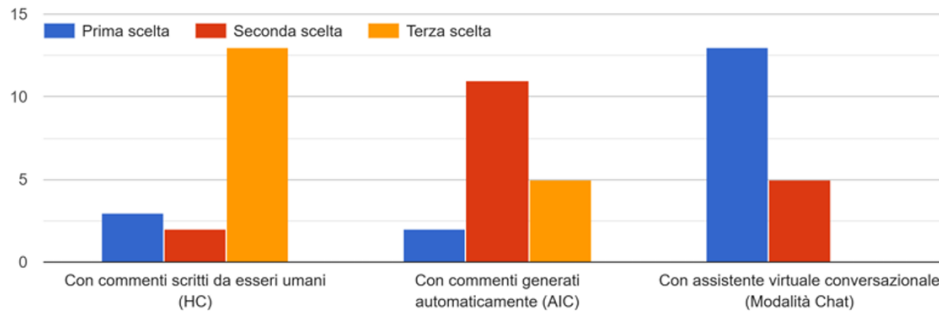


Fig. 9: Favorites mode

5.7.2. Modalità più utile per comprendere il codice

Agli utenti è stato anche chiesto quale delle tre modalità trovassero più utile per comprendere il codice. Ancora una volta, l'assistente virtuale (**Chat**) si è distinto come il più efficace, ricevendo 10 voti come prima scelta su 18 partecipanti. Ciò conferma l'efficacia percepita dell'interazione dinamica nel chiarire concetti e guidare la comprensione.

I **commenti generati automaticamente (AIC)** hanno ricevuto un numero bilanciato di voti come prima scelta, con 6 partecipanti che li hanno selezionati come più utili, suggerendo che, pur essendo meno interattivi, forniscono comunque un supporto significativo in alcuni casi.

Al contrario, i **commenti scritti da esseri umani (HC)** hanno ricevuto solo 2 voti come prima scelta e sono stati spesso classificati come seconda o terza opzione. Questo risultato può sembrare controintuitivo, ma suggerisce che la natura statica o la qualità variabile dei commenti umani possa aver limitato la loro efficacia percepita rispetto alle altre due modalità.

Complessivamente, emerge un quadro coerente: l'interazione diretta con un assistente non solo è stata la modalità più preferita, ma anche quella ritenuta più utile per comprendere il codice.

5.7.3. Risposte aperte a supporto dei risultati

Le risposte aperte confermano che la modalità dell'assistente virtuale è stata percepita come la più utile: molti partecipanti hanno sottolineato come la possibilità di "porre domande specifiche" o ricevere spiegazioni "riga per riga" abbia notevolmente facilitato la comprensione, soprattutto con linguaggi complessi come Rust. Altri hanno apprezzato i **commenti generati automaticamente** per la loro chiarezza e distribuzione all'interno del codice, sebbene alcuni li abbiano trovati poco utili o troppo generici. I **commenti scritti da esseri umani**, invece, sono stati talvolta percepiti come imprecisi o eccessivamente soggettivi.

5.7.4. Analisi delle risposte aperte del questionario finale

Le risposte aperte forniscono ulteriori approfondimenti sui risultati quantitativi riguardanti le preferenze degli utenti nelle diverse modalità di comprensione. Molti partecipanti hanno elogiato la modalità **Chat** per la sua interattività e la possibilità di porre domande specifiche durante la lettura del codice, come evidenziato in commenti del tipo:

- “La modalità chat è più efficiente e veloce, va dritta al punto in modo chiaro e senza digressioni inutili”
- “Poter parlare con Chat aiuta a risolvere dubbi, soprattutto con linguaggi complessi come Rust.”

Questa modalità è stata considerata la più user-friendly, grazie a un’interazione più naturale e flessibile rispetto ai commenti statici.

I **commenti generati automaticamente (AIC)** sono stati generalmente ritenuti chiari e frequenti in diverse parti del codice, offrendo un supporto passo-passo. Tuttavia, alcuni utenti li hanno percepiti come troppo generici o poco approfonditi in certi casi.

I **commenti umani (HC)** hanno ricevuto valutazioni contrastanti: alcuni ne hanno riconosciuto il valore nel trasmettere riflessioni personali e migliorare la leggibilità del codice, mentre altri li hanno trovati meno precisi o più lenti da interpretare rispetto agli approcci basati su IA o interattivi. Alcuni hanno osservato che la natura statica o la qualità variabile dei commenti umani potrebbe aver limitato la loro utilità percepita.

Complessivamente, le risposte aperte evidenziano come l’interazione dinamica e la possibilità di chiarimenti contestuali offerta dall’assistente virtuale rappresentino un vantaggio significativo nella comprensione del codice, soprattutto per gli utenti che devono affrontare linguaggi di programmazione complessi o poco familiari.

6. CONCLUSIONI E LAVORI FUTURI

In questo studio abbiamo condotto uno **studio con utenti** per investigare come diverse forme di documentazione del codice—ossia commenti scritti da esseri umani, commenti generati dall’IA e interazione con un chatbot—influiscono sulla capacità degli studenti universitari di comprendere il codice sorgente. A tal fine, abbiamo sviluppato **CodeInsight**, una piattaforma sperimentale web progettata per fornire frammenti di codice in Java e Rust in condizioni controllate, permettendo di misurare sia le prestazioni oggettive (accuratezza e tempi) sia le percezioni soggettive (usabilità e preferenze).

I risultati hanno rivelato che, sebbene non siano emerse differenze statisticamente significative in termini di accuratezza nelle domande a scelta multipla tra le diverse modalità di supporto, le tendenze descrittive suggeriscono che i **commenti scritti da esseri umani** hanno portato a una comprensione leggermente migliore, mentre i

commenti generati dall'IA hanno permesso tempi di elaborazione più rapidi, specialmente con Java. L'interazione con il **chatbot**, pur essendo associata a tempi di completamento più lunghi, è stata costantemente valutata come la modalità più utile e preferita dai partecipanti, in particolare nel gestire frammenti di codice più complessi in Rust.

Mentre i risultati quantitativi hanno evidenziato tendenze generali nelle prestazioni e nelle preferenze, le risposte aperte offrono una visione più sfumata dell'esperienza dei partecipanti. Molti studenti hanno sottolineato il valore dell'interattività nella modalità Chat, evidenziando la possibilità di porre domande chiarificatrici, ricevere spiegazioni riga per riga e adattare il livello di dettaglio alle proprie esigenze. Alcuni hanno notato che questo era particolarmente vantaggioso con linguaggi meno familiari come Rust, dove i chiarimenti in tempo reale aiutavano a colmare le lacune nella conoscenza pregressa. Altri hanno apprezzato lo stile conciso del chatbot, che evitava ridondanze e rendeva l'interazione più user-friendly.

I **commenti generati automaticamente (AIC)** sono stati spesso descritti come chiari e distribuiti in modo coerente all'interno del codice, permettendo una comprensione passo-passo della logica del programma. Per alcuni partecipanti, questa guida sistematica è risultata particolarmente efficace, anche se altri hanno osservato che le spiegazioni a volte apparivano troppo generiche o insufficientemente dettagliate per compiti di ragionamento complessi.

I **commenti scritti da esseri umani (HC)** hanno ricevuto feedback più contrastanti. Alcuni partecipanti ne hanno riconosciuto il valore nella leggibilità e nelle riflessioni personali che trasmettevano, mentre altri li hanno giudicati imprecisi, di qualità variabile o più lenti da interpretare rispetto agli approcci basati su IA o interattivi. Questa variabilità sembra aver limitato la loro utilità percepita rispetto al supporto più standardizzato dei commenti generati dall'IA o alla flessibilità dell'assistenza interattiva.

Complessivamente, le risposte aperte confermano la conclusione che **l'interazione con il chatbot** non è stata solo la modalità più preferita, ma anche quella più efficace nella pratica, soprattutto in scenari in cui gli studenti affrontavano costrutti di programmazione complessi o poco familiari. Allo stesso tempo, i partecipanti hanno riconosciuto il ruolo complementare dei commenti generati automaticamente nell'offrire una guida strutturata e l'utilità occasionale dei commenti umani quando fornivano spunti chiari e ben argomentati.

Lavori futuri:

- Ampliare il **campione di partecipanti** per migliorare la potenza statistica e la generalizzabilità dei risultati.
- Raffinare la piattaforma **CodeInsight** includendo linguaggi di programmazione più diversi e compiti più complessi, consentendo una valutazione più approfondita di come le modalità di supporto si comportino con l'aumentare della difficoltà dei compiti.

- Integrare e confrontare altri **LLM** (ad esempio Claude, LLaMA, Gemini) per valutare eventuali differenze nella qualità dell'interazione, nella reattività e nell'efficacia pedagogica.
- Adattare il sistema per **contesti didattici** e integrarlo nei corsi di programmazione come strumento per valutazioni formative e apprendimento personalizzato.
- Esplorare modalità **ibride**, combinando commenti statici con agenti di chiarimento interattivi, come possibile direzione per migliorare sia l'usabilità sia i risultati di apprendimento nella comprensione del codice.

Riferimenti

1. S. Stapleton, Y. Gambhir, A. LeClair, Z. Eberhart, W. Weimer, e K. Leach, "A human study of comprehension and code summarization," in *Proceedings of the 28th International Conference on Program Comprehension (ICPC)*, 2020.
2. Y. Wu, Y. Wan, Z. Chu, W. Zhao, Y. Liu, H. Zhang, X. Shi, e P. S. Yu, "Can large language models serve as evaluators for code summarization?" *arXiv preprint*, 2024. [Online]. Disponibile: <https://arxiv.org/abs/2405.14089>
3. J. Leinonen, P. Denny, S. MacNeil, e A. Luxton-Reilly, "Comparing code explanations created by students and large language models," in *Proceedings of the 2023 ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE)*, 2023.
4. W. Sun, Y. Miao, Y. Li, H. Zhang, e Z. Chen, "Source code summarization in the era of large language models," in *Proceedings of the 2025 IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025, in stampa. [Online]. Disponibile: <https://wssun.github.io/papers/2025-ICSE-LLMs4CodeSum.pdf>
5. D. Nam, A. Macvean, V. Hellendoorn, C. Zhang, e K. Leach, "Using an LLM to help with code understanding," in *Proceedings of the 2024 IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024. [Online]. Disponibile: <https://arxiv.org/abs/2402.18225>
6. A. Software, "Opencms core repository," <https://github.com/alkacon/opencms-core/blob/master/src-modules/org/opencms/workplace/CmsLoginHelper.java>, consultato il 01-09-2025.
7. T. Contributors, "Tauri module (mod.rs)," repository framework — ipc, <https://github.com/tauri-apps/tauri/blob/6a39f49991e613e8f3befe0e8dff288482ccdd89/crates/tauri/src/ipc/mod.rs>, consultato il 01-09-2025.

